

Image Filtering

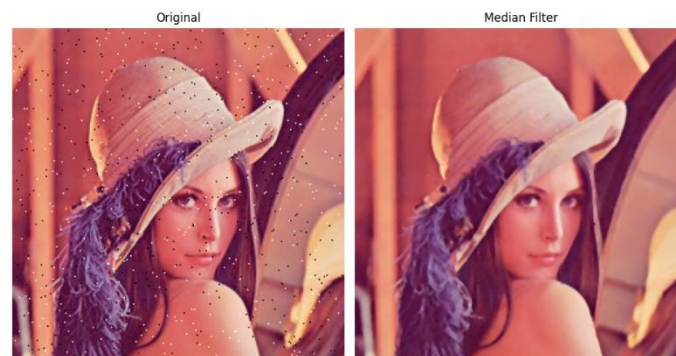
1. Noise Removal

Steps

1. I loaded the noisy image *lena_noise.png*, which contains visible **salt-and-pepper noise**—isolated white and black pixel outliers.
2. To reduce this type of noise, I applied a **median filter** using OpenCV's `cv2.medianBlur()` with a kernel size of **3×3**.
3. The median filter replaces each pixel by the median value in its neighborhood. Because salt-and-pepper noise appears as extreme outliers, the median operation effectively removes them while preserving edges better than linear smoothing.
4. I visualized the original image and the filtered result using Matplotlib.

Results

- The original image shows strong salt-and-pepper noise distributed across the face and background.
- After applying the median filter:
 - The impulsive noise is **significantly reduced**.
 - Major structures (e.g., edges of face, hair, contrast boundaries) are **preserved** because the median filter does not blur edges as strongly as an averaging or Gaussian filter.
- A kernel size of 3 provided good denoising while avoiding excessive smoothing. Larger kernels (for ex. 5 or 7) remove more noise but also begin to soften edges noticeably.



Insights / Discussion

- **Why median filtering works for salt-and-pepper noise**
Median filtering is specifically effective for impulse noise because:
 - Outlier pixels (0 or 255) do **not affect the median** unless they dominate the neighborhood.
- **Conclusion**
The median filter is an appropriate, targeted method for removing salt-and-pepper noise. It successfully cleaned the image while maintaining the important details, making it well-suited for this type of degradation.

2. Hybrid Images

Steps

1. I used two different images for this task.
The aim was to combine the **low frequencies** of the first image with the **high frequencies** of the second image so that one image is visible up close and another dominates from a greater distance.
2. First, I scaled both images to the same size.
Then I used a **Gaussian blur** to create the low-frequency component. This is done by removing finer details through blurring.
3. The high-frequency component of the second image was calculated by subtracting its blurred version from the original image.
This preserves edges and fine details in particular.



4. Then, both components were added pixel by pixel and the value range was limited to valid intensities (clipping).
5. Finally, the results were displayed in different scales to show the typical hybrid effect:
Up close, you mainly see the high frequencies, while from a distance, you see the low frequencies.



Results

- The first image becomes a smooth low-frequency version with only coarse structures.
- The second image contributes high-frequency details like edges and textures.
- Combined, the hybrid effect appears: close up you see high frequencies, from a distance the low-frequency image dominates.
- When scaled down, the low-frequency content becomes even more prominent.

Insights / Discussion

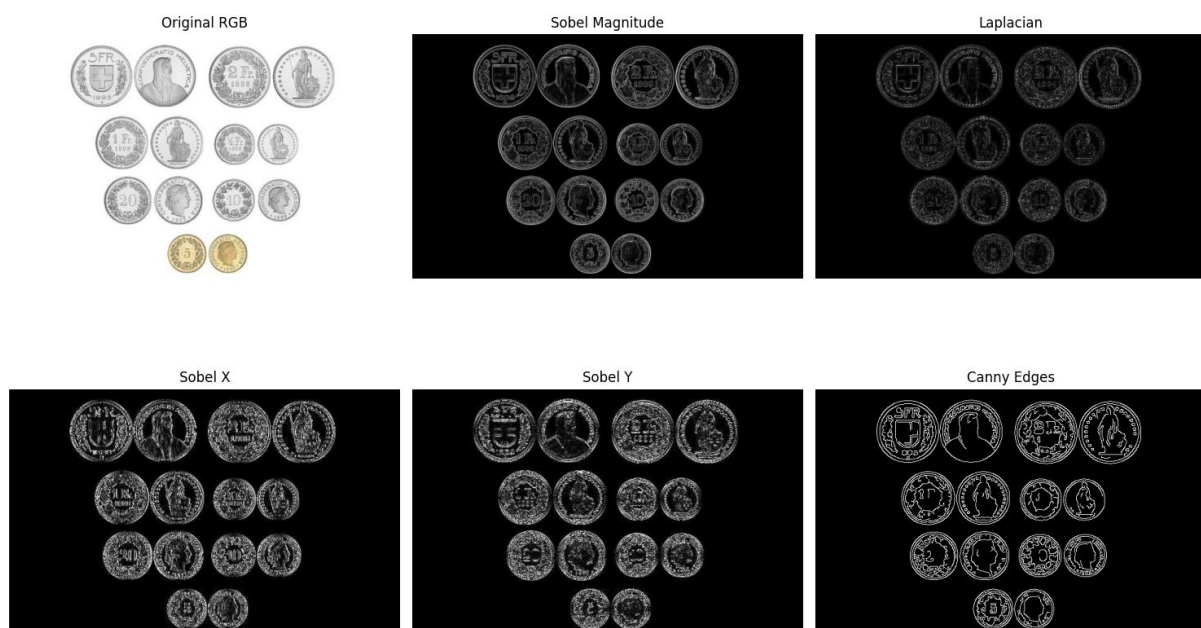
- The Gaussian blur acts as an effective low-pass filter: it suppresses high frequencies such as edges and fine details while keeping only coarse structures.
- High-frequency components are obtained by subtracting the blurred image from the original, a stable and standard method for hybrid images.
- The choice of σ (sigma) is crucial:
 - Too large $\rightarrow$ excessive smoothing, loss of structure in the low-frequency image.
 - Too small $\rightarrow$ insufficient frequency separation, weaker hybrid effect.
- Hybrid images illustrate how the human visual system processes frequencies differently: high frequencies dominate up close, while low frequencies dominate at greater distances.

3. Edge Detection on Coins

Steps

1. I captured an image of one or more coins and converted the image to grayscale for edge detection.
 2. First, I applied **basic gradient filters**:
 - **Sobel operator** in both x- and y-direction.
 - **Laplacian filter**, which detects regions of rapid intensity change
 3. I used these filters to compute gradient responses and visualize the magnitude of edges.
 4. Next, I applied the **Canny edge detector**, which is more advanced and includes several steps:
 - **Gaussian smoothing** to reduce noise
 - **Gradient magnitude and direction** using Sobel filters
 - **Non-maximum suppression** to thin out edges
 - **Double thresholding** to classify strong and weak edge pixels
 - **Hysteresis tracking**, where weak pixels connected to strong ones are kept
- I used OpenCV's implementation, which performs all these steps internally.

Results



- The **Sobel filters** clearly highlight the contours of the coins, especially circular boundaries. However, they also introduce double edges and respond strongly to texture inside the coins.
- The **Laplacian filter** produces sharper but noisier edges. It reacts strongly to small intensity variations, which results in many isolated pixel responses.
- The **Canny detector** provides the cleanest edge map:
 - The coin boundaries are smooth and mostly continuous
 - Noise and small artifacts in the background are successfully removed
 - Thin, well-localized edges are produced due to non-maximum suppression
- Overall, Canny yields a more structured and usable representation of the coin contours than simple gradient filters.

Insights / Discussion

- **Simple gradient filters (Sobel, Laplacian)** detect intensity changes locally, but they:
 - are sensitive to noise,
 - produce thick or double edges,
 - lack any mechanism for suppressing irrelevant weak gradients.
- **Canny**, on the other hand, incorporates multi-stage processing:
 - Noise reduction avoids false detections
 - Non-maximum suppression creates thin, precise edges
 - Double thresholds and hysteresis ensure that real object boundaries are kept together as continuous curves
- **When each method performs better:**
 - **Sobel / Laplacian** are useful for quick edge visualization or when high-frequency texture is desired.
 - **Canny** is preferable for segmentation, object detection, shape analysis, or any task requiring clean, precise, connected contours.

Reflection

- Low-pass filters smooth the image by removing high frequencies, reducing noise but also softening edges.
- High-pass filters enhance edges and fine details but can amplify noise.
- Filtering helps when noise dominates, but too much smoothing removes important structure.
- Sharpening improves detail visibility but risks creating artifacts if the image is noisy.
- There is always a trade-off: stronger noise reduction reduces sharpness, while preserving sharpness keeps more noise.